

Start coding or [generate](#) with AI.

```
!pip install nltk
```

```
import nltk
nltk.download('brown')
nltk.download('universal_tagset')
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/d
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-package
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Unzipping corpora/brown.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Unzipping taggers/universal_tagset.zip.
True
```

```
sentences = brown.sents()
print("Total Sentences:", len(sentences))
```

Total Sentences: 57340

```
tokens = [word.lower() for sent in sentences for word in sent]
print("Total Tokens:", len(tokens))
```

Total Tokens: 1161192

```
def build_ngram(tokens, n):
    model = defaultdict(Counter)
    for i in range(len(tokens)-n+1):
        context = tuple(tokens[i:i+n-1])
        word = tokens[i+n-1]
        model[context][word] += 1
    return model

# Build models
bigram_model = build_ngram(tokens, 2)
trigram_model = build_ngram(tokens, 3)

print("Bigram & Trigram Models Built Successfully")
```

Bigram & Trigram Models Built Successfully

```
def predict_next(context, model):  
    context = tuple(context.lower().split())  
    if context not in model:  
        return []  
    return [word for word, _ in model[context].most_common(5)]
```

```
print("Bigram Prediction for 'government':")  
print(predict_next("government", bigram_model))  
  
print("\nTrigram Prediction for 'the government decided':")  
print(predict_next("the government decided", trigram_model))  
  
print("\nTrigram Prediction for 'he said':")  
print(predict_next("he said", trigram_model))
```

```
Bigram Prediction for 'government':  
['of', '.', ',', 'and', 'was']  
  
Trigram Prediction for 'the government decided':  
[]  
  
Trigram Prediction for 'he said':  
['.', 'he', ':', 'that']
```